

المصفوفات

تُعد المصفوفات واحدة من أهم بنى البيانات في لغة PHP ؛ حيث تتيح تخزين قيم متعددة داخل متغير واحد. وتتميز المصفوفات في هذه اللغة بقدرتها العالية على استيعاب أنواع مختلفة من البيانات؛ مثل النصوص، الأرقام وحتى المصفوفات الأخرى. ويُعد استيعاب كيفية التعامل مع المصفوفات ركيزة أساسية لإدارة البيانات ومعالجتها بكفاءة داخل الأنظمة الرقمية.

ويمكن ذكر خصائص المصفوفات فيما يلي:

- تنوع البيانات: تمتلك المصفوفات القدرة على تخزين قيم من أنواع متباينة (مثل النصوص، الأعداد الصحيحة، الكائنات، أو المصفوفات المتداخلة) ضمن البنية البرمجية ذاتها.
- المرونة في الحجم: تتميز المصفوفات بأنها ذات أحجام ديناميكية؛ أي أنها تتوسع وتتقلص تلقائياً بناءً على حجم البيانات المدخلة دون الحاجة لتحديد مسبق للسعة التخزينية.
- إدارة البيانات المترابطة: تسمح المصفوفات بتجميع القيم المتعددة والمتصلة سياقياً تحت متغير واحد، مما يسهل من عملية تنظيم وتتبع البيانات المترابطة داخل التطبيق التعليمي.
- دعم الدوال المدمجة: توفر لغة PHP مكتبة غنية من الدوال الجاهزة المخصصة للمصفوفات، والتي تتيح عمليات الترتيب، والدمج، والبحث، والمعالجة المتقدمة للبيانات بفعالية عالية.

أنواع المصفوفات في لغة PHP

تُصنف المصفوفات إلى ثلاثة أنواع رئيسية، يخدم كل منها غرضاً بنوياً محدداً في إدارة البيانات:

المصفوفات المفهرسة:

مجموعة من العناصر يتم تعيين رقم فهرس لكل عنصر. وبشكل تلقائي، يبدأ فهرس العنصر الأول من الرقم (0)، بينما يأخذ العنصر الثاني الفهرس (1) وهكذا باقي العناصر، ويساهم هذا الفهرس التسلسلي في تسهيل عملية الوصول إلى عناصر المصفوفة والتحكم بها.

خصائص المصفوفات المفهرسة:

- الفهرسة التسلسلية: (Sequential Indexing) يتم ترقيم العناصر ترتيبياً؛ حيث يبدأ العنصر الأول بالفهرس 0، والثاني بالفهرس 1، ويستمر هذا النمط لكافة العناصر.
- الحجم الديناميكي: (Dynamic Size) تتميز المصفوفات المفهرسة بقدرتها على التوسع أو التقلص تلقائياً دون الحاجة للإعلان عن حجمها مسبقاً، مما يوفر مرونة عالية في إدارة البيانات.
- مرونة أنواع البيانات: (Flexible Data Types) تتيح المصفوفة المفهرسة إمكانية تخزين أنواع مختلفة من البيانات بداخلها، مثل الأعداد الصحيحة (integers)، السلاسل النصية (strings)، القيم المنطقية (booleans)، والأعداد العشرية (floats).

قواعد كتابة العناصر داخل المصفوفة:

- أنواع البيانات: تضمن تخزين جميع البيانات وفي حالة كانت القيم المخزنة نصوص؛ توضع بين علامات تنصيص " " .
- الفواصل: يُستخدم الرمز (,) للفصل بين كل عنصر والآخر.
- الفهرسة التلقائية: بمجرد كتابة العناصر، يقوم مفسر اللغة بإعطاء الرقم 0 للعنصر الأول، والرقم 1 للثاني، وهكذا، دون تدخل من المتعلم.

طرق تعريف وعرض عناصر المصفوفات المفهرسة

يمكن صياغة المصفوفة المفهرسة في PHP بأسلوبين، كلاهما يؤدي نفس الغرض مع اختلاف بسيط في شكل الكود:

1. الكتابة باستخدام الدالة array()

لإنشاء مصفوفة مفهرسة وتخزين مجموعة من القيم بداخلها، نتبع الخطوات التالية:

1. تعريف المتغير: نبدأ بكتابة رمز السهم بـ \$متبوعاً ب اسم المصفوفة مثل \$subjects ليكون هو الوعاء الذي سيحمل البيانات.

2. أداة التعيين: نضع علامة التساوي = لإعلام البرنامج بأننا سنقوم بتخزين قيم داخل هذا المتغير.
3. كتابة كلمة array: نكتب الكلمة المحجوزة array وهي الدالة المسؤولة عن بناء المصفوفة في لغة PHP.
4. فتح الأقواس: نفتح قوساً دائرياً (لنبدأ بوضع العناصر بداخلها).
5. إضافة القيم: نكتب القيم المراد تخزينها (القيمة 1، القيمة 2، وهكذا)، وإذا كانت نصوصاً نضعها بين علامات تنصيص ". "
6. الفصل بين العناصر: نضع علامة الفاصلة , بين كل قيمة والأخرى لتمييز العناصر عن بعضها.
7. إغلاق الأقواس: نغلق القوس الدائري (بعد الانتهاء من إضافة كافة القيم).
8. إنهاء الأمر: نضع الفاصلة المنقوطة ; في نهاية السطر البرمجي لإتمام العملية بنجاح.

```
array(القيمة1, القيمة2, القيمة3, .....); اسم المتغير $
```

• مثال توضيحي:

```
$subjects = array("برمجة", "تكنولوجيا تعليم", "تصميم");
```

2. الصياغة باستخدام الأقواس المربعة []
- تُعرف بالصياغة المختصرة، وهي الأكثر استخداماً في البرمجة الحديثة نظراً لسهولة استخدامها وسرعة كتابته.
1. بدء تعريف المتغير: نبدأ بكتابة رمز السهم ب \$ متبوعاً ب اسم المصفوفة الذي تختارينه (مثلاً: اسم_المصفوفة).
 2. أداة التعيين: نضع علامة التساوي = لتحديد أن هذا المتغير سيحتوي على قيم المصفوفة.
 3. تحديد نوع البناء: وهنا لدينا طريقتان كما يظهر في الصور:
 4. الطريقة الأولى: نكتب كلمة array متبوعة بقوس دائري (.
 5. الطريقة الثانية: نستخدم الأقواس المربعة [مباشرة (وهي الطريقة المختصرة والأحدث).

6. إضافة المحتوى :نكتب القيم المراد تخزينها داخل المصفوفة (مثل: القيمة1، القيمة2).
7. الفصل بين العناصر :نضع علامة الفاصلة ,بين كل قيمة والتي تليها لتمييز العناصر عن بعضها برمجياً.
8. إغلاق الوعاء :نغلق القوس (سواء كان دائرياً (أو مربعاً ([بعد الانتهاء من كتابة كافة القيم.
9. إنهاء الأمر البرمجي :نضع الفاصلة المنقوطة ;في نهاية السطر، وهي ضرورية جداً لإعلام مفسر اللغة بانتهاء العملية.

`$[القيمة1, القيمة2, القيمة3, .....];` = اسم المتغير

• مثال توضيحي:

`$subjects = ["برمجة", "تكنولوجيا تعليم", "تصميم"];`

كيفية الوصول إلى عناصر المصفوفة المفهرسة

"تتم عملية استدعاء أو الوصول إلى البيانات المخزنة داخل المصفوفة المفهرسة من خلال الإشارة إلى رقم الفهرس الخاص بكل عنصر. ومن الناحية البرمجية، يُمثل هذا الرقم "العنوان الرقمي" لموقع المعلومة داخل الذاكرة، مما يسمح للمطور بجلب أي عنصر وتوظيفه في الواجهة التعليمية بمجرد معرفة ترتيبه".

لتحقيق عملية استدعاء العنصر بنجاح، نتبع الخطوات المنطقية التالية:

1. تحديد المصفوفة المستهدفة :نبدأ بكتابة اسم المتغير الذي يحمل المصفوفة مثل `$students`
2. استخدام الأقواس المربعة :نضع الأقواس المربعة []مباشرة بعد اسم المتغير.
3. تحديد الرقم الاستدلالي :نكتب رقم الفهرس الخاص بالعنصر المطلوب ووضعه داخل [].

`$[اسم المتغير]`

• مثال توضيحي:

```
$students = ["أحمد", "سارة", "ياسين", "ليلي"];  
echo $students[0] . "<br>";
```

تعديل عناصر المصفوفة المفهرسة

تتميز المصفوفات بكونها كيانات ديناميكية قابلة للتحديث؛ حيث يمكن تعديل محتوى أي عنصر مخزن مسبقاً من خلال الإشارة إلى رقم الفهرس (Index) الخاص به وتعيين قيمة جديدة له. ومن الناحية العلمية، تبرز أهمية هذه الخاصية في الأنظمة التعليمية عند الحاجة لتحديث بيانات الطالب (مثل تعديل درجة اختبار أو تغيير حالة القيد) دون الحاجة لإعادة بناء المصفوفة بالكامل.

كيفية الكتابة (الخطوات الإجرائية):

لتعديل عنصر داخل المصفوفة، نتبع القواعد المنطقية التالية:

1. تحديد الهدف: نكتب اسم المصفوفة متبوعاً برقم فهرس العنصر المراد تعديله بين أقواس مربعة [].
2. أداة التعيين: نضع علامة التساوي = لإحلال القيمة الجديدة محل القديمة.
3. إسناد القيمة: نكتب القيمة الجديدة (سواء كانت نصاً -توضع بين علامتي تنصيص- أو رقماً) وننهي السطر بالفاصلة المنقوطة .;

القيمة = [] اسم المتغير \$

• مثال توضيحي:

```
$students = ["أحمد", "سارة", "ياسين", "ليلي"];  
$students[0] = "نرمين";
```

التكرار عبر المصفوفات المفهرسة:

تُعد عملية التكرار الآلية الأكثر كفاءة لمعالجة وعرض عناصر المصفوفات المفهرسة، خاصة عندما تحتوي المصفوفة على عدد كبير من البيانات (مثل قوائم أسماء الطلاب في فصل دراسي). وتوفر لغة PHP عدة طرق لتنفيذ ذلك، أبرزها وأكثرها استخداماً هي حلقة foreach؛ نظراً لسهولة استخدامها وقدرتها على التعامل مع المصفوفة ككتلة واحدة دون الحاجة لإدارة عدادات رقمية يدوية.

كيفية الكتابة (المنطق الإجرائي):

1. تحديد المصدر: نبدأ بكلمة foreach متبوعة بقوس دائري (يحتوي على اسم المصفوفة مثل \$students).
2. كلمة العبور: نستخدم الكلمة المحجوزة as لإعلام البرنامج أننا سنقوم بنقل كل عنصر على حدة إلى متغير مؤقت.
3. متغير الاستقبال: نضع متغيراً جديداً مثل (\$name) ليكون هو المستلم للقيمة في كل دورة تكرارية.
4. حيز التنفيذ: نفتح الأقواس { لنضع بداخلها الأوامر التي ستنفذ داخل كل حلقة تكرارية.

```
foreach (اسم متغير الاستقبال $ as اسم المصفوفة $)
{
}
```

• مثال توضيحي:

```
foreach ($students as $name)
{
    echo $name . "<br>";
}
```

إضافة عناصر جديدة للمصفوفة المفهرسة:

تتميز المصفوفات المفهرسة بقدرتها على التوسع الديناميكي؛ حيث يمكن للمطور إضافة عناصر جديدة للمصفوفة في أي وقت باستخدام دالة `array push()` وتكمن أهمية هذه الدالة في قيامها بإضافة العنصر الجديد في نهاية المصفوفة مع تعيين رقم فهرس تلقائي له، مما يضمن استمرارية التسلسل الرقمي دون الحاجة لتدخل يدوي من المبرمج.

كيفية الكتابة (المنطق الإجرائي):

لإضافة قيمة جديدة للمصفوفة، نتبع القواعد المنطقية التالية:

1. استدعاء الدالة: `(array_push)` نبدأ بكتابة اسم الدالة المسؤولة عن عملية الإضافة.
2. تحديد الهدف: نفتح قوساً دائرياً ونضع أولاً اسم المصفوفة التي نريد الإضافة إليها مثل `$students`.
3. تحديد القيمة: نضع فاصلة ، ثم نكتب القيمة الجديدة المراد إضافتها.
4. الإغلاق والإنهاء: نغلق القوس (ونضع الفاصلة المنقوطة ؛

```
;(القيمة الجديدة , اسم المصفوفة)$array_push
```

• مثال توضيحي:

```
$students = ["محمد" , "سارة"] ;  
array_push($students , "ياسين") ;
```

حذف عنصر محدد من المصفوفة المفهرسة

تستخدم دالة `unset()` لحذف أي عنصر من المصفوفة بناءً على رقم الفهرس الخاص به، ولكن يجب الانتباه إلى أن هذه الدالة تحذف العنصر وفهرسه تماماً، لكنها لا تقوم بإعادة ترتيب الأرقام المتبقية تلقائياً، مما يتطلب أحياناً خطوة إضافية لإعادة تنظيم المصفوفة.

كيفية الكتابة (المنطق الإجرائي):

1. استدعاء الدالة: unset.

2. تحديد الهدف بدقة: نضع اسم المصفوفة ورقم الفهرس المراد حذفه بين أقواس مربعة مثلاً:

`$students[1]`

3. إعادة الفهرسة (خطوة اختيارية): إذا أردنا أن تعود الأرقام متسلسلة (0, 1, 2...) بدون فراغات،

نستخدم دالة `array_values()`.

لحذف: `unset([رقم الفهرس للعنصر المراد حذفه] اسم المصفوفة);`

لإعادة الترتيب: `اسم المصفوفة = array_values(اسم المصفوفة);`

• مثال توضيحي:

```
$students = ["أحمد", "سارة", "ياسين"];  
unset($students[1]);  
$students = array_values($students);
```

المصفوفات الترابطية (PHP Associative Arrays)

تُمثل المصفوفة الترابطية بنية بيانات خاصة في لغة PHP، حيث يتم تخصيص اسم أو "تسمية مرجعية" لكل عنصر بدلاً من الاعتماد على المؤشرات الرقمية التقليدية. فبينما تعتمد المصفوفات الاعتيادية على الأرقام لتحديد مواقع العناصر (حيث يشغل العنصر الأول الموضع 0، والثاني الموضع 1، وهكذا)، تعود المصفوفات الترابطية لاستخدام الكلمات أو الأسماء كأدوات استدلالية للوصول إلى البيانات.

وتعتمد هذه المصفوفات على مفهوم "المفاتيح: (Keys)" بدلاً من استدعاء العنصر بناءً على ترتيبه العددي (مثلاً: "أعطني العنصر في الموضع 0")، نستخدم اسماً دالاً للوصول للقيمة المطلوبة (مثلاً: "أعطني القيمة المرتبطة بمفتاح 'الاسم'" أو "القيمة التابعة لمفتاح 'العمر'").

عناصر المصفوفة الترابطية:

- المفتاح (Key): يُمثل المعرف الفريد للعنصر) مثل("name" :، ويُستخدم للوصول للقيمة بدلاً من الرقم التلقائي.
- عامل الربط: (=>) هو الرمز المسؤول عن تعيين القيمة للمفتاح المحدد، ويُسمى برمجياً عامل "يؤدي إلى".
- القيمة (Value): هي البيانات الفعلية المراد تخزينها، وقد تكون نصاً، عدداً، أو أي نوع آخر من البيانات؛ في حالة كانت القيم نصاً فلا بد من وضعها داخل علامتي تنصيص.

كيفية إنشاء المصفوفات الترابطية

لإنشاء مصفوفة ترابطية، يتم الاعتماد على الدالة التقليدية `array()` أو استخدام الصياغة المختصرة الحديثة `[]`. وتعتمد عملية البناء الهيكلي لهذه المصفوفة على ربط المفاتيح بـ القيم المقابلة لها باستخدام عامل التعيين `(=>)`.

1. الصياغة باستخدام الدالة `array()`

```
$ = array اسم المصفوفة  
(  
    القيمة => "العنصر1",  
    القيمة => "العنصر2",  
    القيمة => "العنصر3",  
    وهكذا باقي العناصر  
);
```

• مثال توضيحي:

```
$student = array  
(  
    "name" => "أحمد",  
    "age" => 25,  
    "grade" => "امتياز"  
);
```

2. الصياغة المختصرة

```
[ اسم المصفوفة$  
  
    القيمة => "العنصر1",  
    القيمة => "العنصر2",  
    القيمة => "العنصر3",  
    وهكذا باقي العناصر  
];
```

- مثال توضيحي:
مثال للتعامل مع عناصر المصفوفة الترابطية :

```
$student =  
[  
    "name" => "أحمد",  
    "age"  => 25,  
    "grade" => "امتياز"  
];
```

```
$person =  
[  
    "name" => "نرمين",  
    "age"  => 23,  
    "city" => "القاهرة"  
];  
  
echo $person["name"] . "<br>";  
echo $person["age"] . "<br>";  
echo $person["city"] . "<br>";
```

استخدام حلقة التكرار foreach لمعالجة المصفوفات

"تُعد حلقة التكرار foreach الوسيلة البرمجية الأكثر كفاءة وسهولة للتنقل عبر عناصر المصفوفات الترابطية؛ حيث تمنح المطور القدرة على الوصول المباشر لكل من المفتاح والقيمة في كل دورة تكرارية بشكل آلي. ومن الناحية العلمية، تكمن أهمية هذه الحلقة في قدرتها على معالجة كافة عناصر المصفوفة بغض النظر عن طولها، دون الحاجة لاستخدام عداد يدوي، مما يقلل من احتمالية حدوث أخطاء برمجية أثناء التنفيذ.

كيفية الكتابة:

لتحقيق عملية العرض الآلي للبيانات، نقوم بصياغة جملة التكرار بناءً على القواعد المنطقية التالية:

- الكلمة الافتتاحية: (foreach) نبدأ بكتابة هذه الكلمة المحجوزة، وهي تعني "لكل عنصر"، حيث تعطي أمراً للمحرك البرمجي بالمرور على كافة محتويات المصفوفة.
- قوس تحديد المصدر: (()) نفتح قوساً دائرياً لنحدد بداخله الوعاء الذي يحتوي على البيانات؛ وفي كودنا هذا نضع اسم المصفوفة التي أنشأناها وهي \$student_profile.
- كلمة التفكيك: (as) تلي اسم المصفوفة، ووظيفتها إخبار البرنامج بكيفية "تفكيك" كل عنصر داخل المصفوفة عند المرور عليه.
- متغيرات الاستقبال * : (\$key => \$value) نضع أولاً متغيراً يسمى \$key ليستقبل المفتاح (مثل : id). أو name.
- نضع سهم الربط => ليفصل بين المفتاح والقيمة.
- نضع متغيراً ثانياً يسمى \$value ليستقبل المعلومة الفعلية المخزنة (مثل: "أحمد" أو "202601").
- قوس إغلاق الإعداد: (()) نغلق القوس الدائري لنعلن انتهاء مرحلة إعداد مسار البيانات وبدء مرحلة التنفيذ.
- أقواس حيز التنفيذ: ({}).
- نفتح القوس المجدد {ليمثل "بداية" الأوامر التي ستتكرر.
- نكتب بداخلها أمر الطباعة echo الذي يستخدم المتغيرين \$key و \$value لعرض النتائج.
- نغلق القوس المجدد }ليمثل "نهاية" جملة التكرار، وبمجرد إغلاقه سيعود البرنامج آلياً للعنصر التالي في المصفوفة حتى ينتهي منها جميعاً.

```
foreach ( as $key => $value اسم المصفوفة )
{
    echo "<b>" . $key . "</b> : " . $value . "<br>";
}
```

- مثال توضيحي:

```

$student_profile =
[
    "id"      => 202601,
    "name"    => "أحمد محمد محمود",
    "level"   => "المستوى الثالث",
    "gpa"     => 3.85,
    "email"   => "ahmed@edu.com"
];

// استخدام حلقة التكرار لطباعة المفاتيح الإنجليزي وقيمته العربية
foreach ($student_profile as $key => $value) {
    echo "<b>" . $key . "</b> : " . $value . "<br>";
}

```

استخدام حلقة التكرار (for) مع مفاتيح المصفوفات

"بالرغم من سهولة حلقة foreach في التعامل مع المصفوفات، إلا أن استخدام حلقة for التقليدية يمنح المطور تحكماً أدق في ترتيب المعالجة. ومن الناحية العلمية، تعتمد هذه الطريقة على استخراج المفاتيح أولاً كمصفوفة مستقلة، ثم المرور عليها باستخدام عداد رقمي يدوي. وتكمن أهمية هذا الأسلوب في الحالات التي نحتاج فيها للتحكم في عدد مرات التكرار بدقة، أو عند الرغبة في الوصول للعناصر بناءً على ترتيبها العددي رغم كونها مصفوفة ترابطية".

كيفية الكتابة:

لتحقيق عملية العرض باستخدام حلقة for اليدوية، نقوم بصياغة الجمل البرمجية بناءً على القواعد المنطقية التالية:

- استخراج المفاتيح: (array_keys) نستخدم هذه الدالة المدمجة لتحويل أسماء الحقول (المفاتيح) إلى قائمة رقمية مرتبة ونخزنها في متغير مثل \$keys.
- الكلمة الافتتاحية: (for) نبدأ بكتابة الكلمة المحجوزة لبدء حلقة التكرار التي تعتمد على العداد الرقمي.

- تجهيز العداد: (\$i = 0) داخل القوس الدائري، نبدأ بتعريف متغير العداد (Index) ونعطيه القيمة صفر ليمثل نقطة الانطلاق.
- شرط الاستمرار: (\$i < count(\$keys)) نحدد شرطاً يضمن استمرار الحلقة ما دام العداد أقل من إجمالي عدد العناصر المتاحة في مصفوفة المفاتيح.
- معامل الزيادة: (\$i++) نأمر العداد بالزيادة بمقدار واحد بعد كل دورة للانتقال للعنصر التالي.
- استعادة المفتاح والقيمة: داخل الأقواس المربعة: { }
- نستخرج اسم المفتاح الحالي من مصفوفة المفاتيح باستخدام العداد. \$keys[\$i] :
- نستخدم هذا المفتاح للوصول للقيمة المقابلة له في المصفوفة الأصلية.
- أمر الطباعة: (echo) نقوم بعرض النتائج النهائية التي تم استخراجها يدوياً.

```
$keys = array_keys($student_profile);

for ($i = 0; $i < count($keys); $i++)
{
    $key = $keys[$i];
    echo "<b>" . $key . "</b> : " . $student_profile[$key] . "<br>";
}
```

- مثال توضيحي:

```
$student_profile =
[
    "id"      => 202601,
    "name"    => "أحمد محمد محمود",
    "level"   => "المستوى الثالث",
    "gpa"     => 3.85,
    "email"   => "ahmed@edu.com"
];

$keys = array_keys($student_profile);
for ( $i = 0; $i < count($keys); $i++)
{
    $key = $keys[$i];

    $value = $student_profile[$key];

    echo "<b>" . $key . "</b> : " . $value . "<br>";
}
```

التحقق من وجود المفتاح في المصفوفة:

عند التعامل مع الأنظمة التعليمية، قد يتطلب الأمر استدعاء حقل معين (مثل رقم الهاتف أو درجة اختبار)، ولضمان استقرار الكود وتجنب ظهور أخطاء تقنية، يجب التأكد من وجود المفتاح قبل البدء بمعالجته. ويتم ذلك باستخدام دالة `array_key_exists()` أو `isset()`، مما يساهم في بناء برمجيات أكثر أماناً وقدرة على التعامل مع البيانات المفقودة.

كيفية الكتابة:

لتحقيق عملية الفحص البرمجي، نتبع الخطوات المنطقية التالية:

- استخدام الجملة الشرطية: `(if)` نبدأ بكتابة `if` لفتح اختبار شرطي يتأكد من الحالة قبل التنفيذ.
- دالة الفحص: `(array_key_exists)` نضع داخل القوسين اسم المفتاح الذي نبحث عنه (مثل `"gpa"` ثم اسم مصفوفة الطالب `$student_profile`).
- تحديد مسار النجاح: نفتح القوس المجدد `{` لنكتب بداخله الأوامر التي ستنفذ فقط في حال كان المفتاح موجوداً.
- تحديد مسار البديل: `(else)` نستخدمها لإخبار البرنامج بما يجب فعله في حال عدم العثور على المفتاح (مثل إظهار رسالة "البيان غير متوفر").

طريقة الكتابة:

```
if (array_key_exists("اسم المفتاح", $المصفوفة))
{
    التنفيذ في حالة التحقق
}
else
{
    التنفيذ في حالة عدم التحقق
}
```

- مثال توضيحي:

```
$student_profile =  
[  
    "id"      => 202601,  
    "name"    => "أحمد محمد محمود",  
    "level"   => "المستوى الثالث"  
    // نلاحظ هنا عدم وجود مفتاح الـ gpa  
];  
  
if (array_key_exists("gpa", $student_profile))  
{  
    echo "المعدل التراكمي للطالب هو " . $student_profile["gpa"];  
}  
else  
{  
    echo ".عذراً، المعدل التراكمي لهذا الطالب غير مسجل حالياً";  
}
```

حذف العناصر من المصفوفة الترابطية (Removing Elements)

في البيئات التعليمية الرقمية، قد نحتاج إلى حذف بيان معين من سجل الطالب (مثل كلمة مرور مؤقتة أو حقل لم يعد مطلوباً)، وتوفر لغة PHP دالة `unset()` للقيام بهذه المهمة. ومن الناحية العلمية، تعمل هذه الدالة على إزالة المفتاح والقيمة المرتبطة به تماماً من ذاكرة النظام، مما يساهم في تحسين أداء المنصة وتأمين البيانات غير الضرورية.

كيفية الكتابة:

لتحقيق عملية الحذف البرمجي، نتبع القواعد المنطقية التالية:

- استدعاء الدالة: `unset()` نبدأ بكتابة الكلمة المحجوزة المسؤولة عن عملية الحذف.
- تحديد الهدف: نفتح قوساً دائرياً ونضع بداخله اسم المصفوفة `$student_profile`.
- تحديد المفتاح: نستخدم الأقواس المربعة `[]` وبداخلها نكتب اسم المفتاح المراد حذفه (مثل `"email"`).

- إنهاء الأمر :نغلق القوس الدائري ونضع الفاصلة المنقوطة ؛ليعرف المفسر أن عملية الإزالة قد تمت بنجاح.
طريقة الكتابة :

```
["اسم المفتاح"] (اسم المصفوفة)$unset;
```

- مثال توضيحي:

```
$student_profile =
[
    "id"      => 202601,
    "name"    => "أحمد محمد محمود",
    "email"   => "ahmed@edu.com"
];

unset($student_profile["email"]);
```

متى نستخدم المصفوفات الترابطية؟(When to Use Associative Arrays)

تُعد المصفوفات الترابطية خياراً مثالياً في المواقف البرمجية والتعليمية التالية:

- ربط القيم بمفاتيح ذات دلالة ومعنى :عندما تحتاجين لربط البيانات بمسميات واضحة تسهل فهم المحتوى (مثل: بيانات مستخدم، تفاصيل منتج تعليمي، أو سجلات طلاب).
- الوصول للعناصر من خلال المسمى بدلاً من الموقع :في الحالات التي يكون فيها استدعاء العنصر باسمه (مثل : الاسم ("أكثر دقة ومنطقية من استدعائه برقم موقعه (مثل. 0) :
- تمثيل البيانات المهيكلة ببساطة :عندما ترغبين في تنظيم بيانات معقدة ومرتبطة ببعضها البعض دون الحاجة للدخول في تعقيدات بناء "الأصناف (Classes) "أو "الكائنات.(Objects) "

المصفوفات متعددة الأبعاد (Multidimensional Arrays)

تُعد المصفوفات متعددة الأبعاد بمثابة مصفوفات متداخلة؛ حيث تقوم بتخزين مصفوفات أخرى كعناصر بداخلها. وتزداد درجة تعقيد هذه المصفوفات مع كل 'بعد' إضافي يتم إلحاقه بها، مما يتطلب استخدام عدة مؤشرات للوصول إلى العنصر المطلوب. وتعتبر هذه المصفوفات مثالية لتنظيم البيانات المعقدة والمهيكلية، ومن أشهر أشكالها المصفوفات ثنائية الأبعاد (التي تشبه الجداول) والمصفوفات ثلاثية الأبعاد.

مفهوم الأبعاد:

تشير الأبعاد في المصفوفات متعددة الأبعاد إلى عدد المؤشرات أو الأرقام الاستدلالية اللازمة لتحديد واختيار عنصر معين. فعلى سبيل المثال، نحتاج في المصفوفة ثنائية الأبعاد إلى مؤشرين (رقم للصف ورقم للعمود) للوصول إلى القيمة المستهدفة. أما في المصفوفات ثلاثية الأبعاد أو الأكثر تعقيداً، فإن كل بُعد إضافي يضيف طبقة جديدة من العمق، مما يتطلب مؤشراً إضافياً للوصول إلى القيمة المستهدفة.

ويساهم هذا النظام الإحداثي المتعدد في تمكين المتعلم من تنظيم البيانات المعقدة والمستمرة في هياكل منطقية مرتبطة تتجاوز الجداول البسيطة لتشمل تمثيل البيانات الهيكلية الضخمة. ويُعد الالتزام ببداية الترقيم في هذه المؤشرات برمجياً من الرقم صفر بشكل افتراضي ركيزة أساسية لضمان دقة استدعاء البيانات واستقرار الأنظمة. هذا الترابط الهيكلي يمنح المتعلم القدرة على تخزين مصفوفات كاملة كعناصر داخل مصفوفة أكبر، مما يخلق بناءً معلوماتياً عميقاً يسهل التعامل مع مجموعات البيانات المتداخلة والضخمة بكفاءة عالية.

المصفوفات ثنائية الأبعاد (Two-dimensional Array)

تعتبر المصفوفة ثنائية الأبعاد أبسط أشكال المصفوفات المتعددة، ويمكن إنشاؤها من خلال وضع مصفوفة داخل مصفوفة أخرى. وبالرغم من قدرة هذا النوع على تخزين مختلف أنواع البيانات، إلا أن الفهرس المستخدم للوصول إليها يكون دائماً رقماً، ويبدأ تلقائياً من الصفر.

	العمود 0	العمود 1	العمود 2
الصف 0	[0][0]	[0][1]	[0][2]

الصف 1	[1][0]	[1][1]	[1][2]
الصف 2	[2][0]	[2][1]	[2][2]

كيفية الكتابة (بناء مصفوفة ثنائية الأبعاد):

1. تعريف المصفوفة ثنائية الأبعاد: نبدأ بكتابة اسم المتغير العام (مثل \$users ثم علامة التساوي = متبوعة بكلمة array لفتح الوعاء الرئيسي للبيانات.
2. إنشاء الصفوف (المصفوفات الفرعية): داخل الأقواس الرئيسية، نكتب كلمة array مرة أخرى لكل صف جديد نريد إنشاؤه، ليمثل كل منها سجلاً مستقلاً (مثل بيانات طالب واحد).
3. إضافة العناصر الداخلية: نفتح قوساً داخلياً لكل صف ونضع بداخله القيم المرتبطة ببعضها (مثل "الاسم" و"الموقع")، مع وضع النصوص بين علامات تنصيص.
4. الفصل بين السجلات: نستخدم الفاصلة ، للفصل بين كل مصفوفة فرعية (صف) والتي تليها، لتمييز حدود كل سجل بيانات داخل الجدول الكبير عدا آخر مصفوفة لا ننهايها بفاصلة.
5. إغلاق الهيكل وإنهاء الأمر: نغلق القوس الرئيسي للمصفوفة ثنائية الأبعاد ونضع الفاصلة المنقوطة ; لإعلام المفسر باكتمال بناء الجدول بنجاح.

```

array = اسم المصفوفة ثنائية الأبعاد
(
    array(عنصر1، عنصر2، عنصر3،.....)،
    array(عنصر1، عنصر2، عنصر3،.....)،
    array(عنصر1، عنصر2، عنصر3،.....)
);

```

• مثال توضيحي:

تتجلى القيمة العملية للمصفوفات ثنائية الأبعاد في القدرة على تنظيم البيانات المرتبطة ببعضها البعض في هيكل واحد متكامل؛ وفي هذا المثال، يتم إنشاء مصفوفة ثنائية الأبعاد مخصصة لتخزين "الأسماء والمواقع".

حيث تعمل المصفوفة الأم كوعاء تنظيمي يحتوي على مصفوفات فرعية متداخلة (Nested Arrays) ، يمثل كل منها سجلاً ببياناتي مستقلاً يجمع بين اسم الشخص وموقعه الجغرافي.

```
$users = array
(
    array("أحمد محمد", "القاهرة"),
    array("سارة محمود", "المنيا"),
    array("ياسين علي", "الإسكندرية")
);
```

المصفوفات الترابطية ثنائية الأبعاد (2D Associative Arrays)

تُعد المصفوفة الترابطية تطوراً هيكلياً للمصفوفات الرقمية التقليدية؛ فبينما تعتمد المصفوفات الرقمية التقليدية على نظام تخزين خطي يعتمد على الأرقام الاستدلالية المتسلسلة (0, 1, 2...) ، تتميز المصفوفة الترابطية بقدرتها على تخصيص "مفاتيح" (Keys) نصية من نوع (String) لكل قيمة مخزنة. ويعمل هذا المفتاح بمثابة معرف دقيق ومباشر يسهل عملية قراءة الكود البرمجي وفهم طبيعة البيانات، مما يحول المتغير من مجرد وعاء للقيم إلى بناء معلوماتي ذاتي التفسير.

وعند الانتقال إلى مستوى البنية ثنائية الأبعاد، يتم توظيف هذه المفاتيح النصية في كلا البعدين (أو أحدهما)، مما يحول المصفوفة إلى بناء هيكلي يُحاكي عمل "قواعد البيانات المصغرة". وفي هذا السياق، يمثل البعد الأول (المفتاح الرئيسي) السجل الكامل، بينما يمثل البعد الثاني (المفاتيح الفرعية) السمات أو الحقول المرتبطة بهذا السجل. فعلى سبيل المثال، يمكن إنشاء مصفوفة لتخزين "درجات الطلاب"، حيث يُستخدم اسم الطالب كمفتاح رئيسي، وتُستخدم أسماء المواد الدراسية كمفاتيح فرعية، لتتقاطع هذه المفاتيح وتحدد بدقة درجة الطالب في مادة معينة.

وتبرز الأهمية لهذه البنية كأداة محورية لضبط مدخلات الأنظمة البرمجية وتجهيزها للمعالجة المعقدة؛ حيث تتيح للمتعلم صياغة أكواد أكثر دقة وموثوقية من خلال فرض مسميات منطقية على البيانات قبل التنفيذ.

ويساهم هذا التنظيم في تعزيز استقرار الأنظمة التعليمية عبر منع حدوث أخطاء "التداخل الرقمي (Index Confusion)، ويضمن معالجة البيانات المسترجعة من قواعد البيانات بشكل يتوافق مع المنطق البرمجي، مما يقلل من احتمالية حدوث أخطاء منطقية عند التعامل مع سجلات الطلاب المتعددة والمتنوعة داخل بيئة التعلم.

طريقة الكتابة المصفوفة الترابطية ثنائية الأبعاد:

1. تعريف المتغير العام (Declaration) نقوم بتعريف المتغير الذي س يحمل المصفوفة؛ نبدأ بوضع

علامة الـ \$ ثم نكتب اسم المتغير مباشرة يليه معامل الإسناد =، ثم نكتب كلمة **array** ونفتح قوساً هلالياً (لتبدأ المصفوفة الأم).

• مثال `$marks = array(`

2. إنشاء المفتاح الرئيسي (Primary Key)

داخل القوسين، نحدد "المفتاح" الذي يمثل السجل الأول (مثل اسم الطالب). نكتب الاسم بين علامتي تنصيص "**Ahmed**"، ثم نضع علامة الربط => (السهم) والتي تتكون من علامة "يساوي" متبوعة بعلامة "أكبر من". هذا السهم يخبر البرمجية بأن هذا الاسم "يشير إلى" محتوى معين.

• مثال `"Ahmed" =>` :

3. بناء المصفوفة الفرعية (Nested Array)

بعد السهم مباشرة، نكتب كلمة **array** مرة أخرى ونفتح قوساً هلالياً جديداً (هذه الخطوة تعني أن القيمة المرتبطة بـ "**Ahmed**" ليست مجرد رقم أو نص، بل هي مصفوفة أخرى تحتوي على تفاصيل إضافية (وهنا يظهر البعد الثاني للمصفوفة).

4. إدخال أزواج البيانات (Key-Value Pairs)

داخل المصفوفة الفرعية، نكرر عملية الربط؛ نضع أسماء العناصر بين علامتي تنصيص " "، ثم سهم الربط =>، ثم نضع القيمة الرقمية للعنصر بعد قيمة كل عنصر يجب وضع علامة الفاصلة ، لنخبر البرنامج بوجود عنصر آخر قادم، ولابد من مراعاة حين تكون القيمة نصية لابد من وضعها داخل علامتي تنصيص.

• مثال، 95 => "Programming" :

5. إغلاق البناء البرمجي (Closing)

بعد الانتهاء من إدخال بيانات المصفوفة الفرعية، نقوم بإغلاق قوس المصفوفة الفرعية بوضع (متبوعة بفاصلة ، (إذا كان هناك مصفوفة أخرى). وفي نهاية الكود تماماً، نقوم بإغلاق قوس المصفوفة الأم بوضع) ثم علامة الفاصلة المنقوطة ؛ لإنهاء السطر البرمجي.

```
array = اسم المصفوفة الترابطية تنائية الأبعاد $
(
    // المصفوفة الفرعية الاولى
    array(
        "القيمة" => "العنصر الأول",
        "القيمة" => "العنصر الثاني",
        "القيمة" => "العنصر الثالث",
        .....
    ),
    (
        // المصفوفة الفرعية الثانية
        array(
            "القيمة" => "العنصر الأول",
            "القيمة" => "العنصر الثاني",
            "القيمة" => "العنصر الثالث",
            .....
        ),
        (
            // المصفوفة الفرعية الثالثة
            array(
                "القيمة" => "العنصر الأول",
                "القيمة" => "العنصر الثاني",
                "القيمة" => "العنصر الثالث",
                .....
            ),
        );
);
```

مثال توضيحي:

```
$marks = array
(
    "Ahmed" => array(
        "Programming" => 95,
        "Operating_Systems" => 85,
        "Databases" => 74,
    ),
    "Khaled" => array
    (
        "Programming" => 78,
        "Operating_Systems" => 98,
        "Databases" => 46,
    ),
    "Yasser" => array
    (
        "Programming" => 88,
        "Operating_Systems" => 46,
        "Databases" => 99,
    ),
);
?>
```

خطوات إنشاء مصفوفة ثلاثية الأبعاد في PHP

تعتمد المصفوفة ثلاثية الأبعاد على نفس مبدأ المصفوفة ثنائية الأبعاد، ولكن مع زيادة مستوى التعقيد في "التداخل" (Nesting).

1. تعريف المتغير العام (The Main Variable)

نقوم بتعريف المتغير الذي سيحتوي على المصفوفة بالكامل؛ نبدأ بوضع علامة (\$) (Dollar Sign)، ثم نكتب اسم المتغير (مثل **myarray**)، يليه معامل الإسناد =، ثم نكتب كلمة **array** ونفتح القوس الهلالي الأول (الذي يمثل الوعاء الرئيسي للمصفوفة).

2. إضافة البُعد الأول (The First Dimension)

داخل القوس الأول، نقوم بفتح مصفوفة جديدة باستخدام كلمة **array** وقوس هاللي آخر .(في المصفوفة ثلاثية الأبعاد، هذا البُعد يمثل "المستوى" أو "الطبقة" (Layer) التي ستحتوي بداخلها على مصفوفات أخرى.

3. إضافة البُعد الثاني (The Second Dimension)

داخل مصفوفة "المستوى"، نقوم بفتح مصفوفة ثالثة (**array**) لتمثيل البُعد الثاني (الذي يعبر غالباً عن "الصفوف" في الجداول). تذكرني أنه كلما زاد عدد الأبعاد، زاد عدد الأقواس المتداخلة. (Nested Braces)

4. إضافة البُعد الثالث والعناصر (The Third Dimension & Elements)

داخل مصفوفة "الصف"، نضع القيم الفعلية أو مصفوفة رابعة تحتوي على البيانات. للوصول إلى عنصر معين في هذا الهيكل، سنحتاج إلى ثلاثة أرقام استدلالية: (Indices)

- المؤشر الأول: لتحديد الطبقة.
- المؤشر الثاني: لتحديد الصف.
- المؤشر الثالث: لتحديد العمود (العنصر المحدد).

5. إغلاق الهيكل البرمجي بدقة (Closing)

بسبب كثرة الأقواس، يجب إغلاق كل مصفوفة فرعية بقوسها الخاص (متبوعة بفاصلة ,لانتقال للعنصر التالي. وفي النهاية المطلقة، نغلق القوس الرئيسي للمصفوفة الأم بـ (ثم نضع الفاصلة المنقوطة ;لإنهاء السطر البرمجي.

```

$اسم_المصفوفة_ثلاثية_الأبعاد = array(
    array(// المستوى الأول
        // المصفوفة الفرعية (الصف الأول داخل المستوى الأول)
        "اسم المفتاح الرئيس" => array(
            "القيمة" => "العنصر الأول",
            "القيمة" => "العنصر الثاني",
            .....
        ),
        // المصفوفة الفرعية (الصف الثاني داخل المستوى الأول)
        "اسم المفتاح الرئيس" => array(
            "القيمة" => "العنصر الأول",
            "القيمة" => "العنصر الثاني",
            .....
        ),
    ),
    array(// المستوى الثاني
        // المصفوفة الفرعية (الصف الأول داخل المستوى الثاني)
        "اسم المفتاح الرئيس" => array(
            "القيمة" => "العنصر الأول",
            "القيمة" => "العنصر الثاني",
            .....
        ),
        // المصفوفة الفرعية (الصف الثاني داخل المستوى الثاني)
        "اسم المفتاح الرئيس" => array(
            "القيمة" => "العنصر الأول",
            "القيمة" => "العنصر الثاني",
            .....
        ),
    ),
);

```

مثال توضيحي:

```

$College = array(
  "Students_data" => array(
    "Student_Profile" => array(
      "Name" => "Ahmed_Ali",
      "ID" => "2026001",
      "Level" => "Fourth_Year"
    ),
    "Student_Courses" => array(
      "Course_1" => "PHP_Programming",
      "Course_2" => "MySQL_Databases",
      "Course_3" => "Web_Development"
    )
  ),
  "Departments_data" => array(
    "Department_Info" => array(
      "Dept_Name" => "Educational_Technology",
      "Head_Name" => "Dr_Khaled",
      "Location" => "Building_A"
    ),
    "Department_subject" => array(
      "Core_Subject" => "Instructional_Design",
      "Practical_Subject" => "Programming_Lab",
    )
  )
);

```

طرق الوصول إلى عناصر المصفوفة المتعددة الأبعاد

تعتمد لغة PHP بشكل أساسي على ثلاث طرق رئيسية للوصول إلى البيانات المخزنة داخل المصفوفات المتداخلة:

1. الوصول المباشر باستخدام الأبعاد

تُعد هذه الطريقة هي الأسرع عندما نكون على علم بالفتاح (Key) أو الفهرس (Index) المحدد. يتم الوصول للعنصر بكتابة اسم المصفوفة متبوعاً بالأبعاد المطلوبة داخل أقواس مربعة [].

1. استدعاء المتغير الرئيسي (\$) (\$)

نضع علامة \$ ثم نكتب اسم المصفوفة الكبرى ؛ لكي يوجه البرنامج البحث داخل وعاء بيانات الكلية.

2. فتح البُعد الأول

نضع قوساً مربعاً '[' ']' ونكتب بداخله المفتاح الرئيسي للمصفوفة .

3. الانتقال للبُعد الثاني

نضع قوساً مربعاً ثانياً '[' ']' ونكتب بداخله المفتاح الفرعي للمصفوفة

4. الوصول للبُعد الثالث

نضع قوساً مربعاً ثالثاً '[' ']' ونكتب بداخله اسم العنصر المراد استخراجه؛ وهو المفتاح الذي يشير مباشرة إلى القيمة المطلوبة.

5. إنهاء السطر البرمجي (؛)

نضع الفاصلة المنقوطة (Semicolon) في نهاية السطر؛ لإعلام محرك PHP بانتهاء هذا الأمر البرمجي بنجاح.

```
$College = array(  
    'Dep_Layer' => array(  
        'Dep_Info' => array(  
            'Dep_Name' => 'تكنولوجيا التعليم',  
            'Head_Name' => 'خالد'  
        )  
    )  
);  
echo $College['Dep_Layer']['Dep_Info']['Head_Name'];
```

2. استخدام حلقة foreach (Foreach Loop)

تُعد الطريقة الأكثر شيوعاً وعملية في PHP ، خاصة عند التعامل مع المصفوفات الترابطية (Associative Arrays)، لأنها تتيح المرور على كافة العناصر دون الحاجة لمعرفة عددها أو مفاتيحها مسبقاً.

- الآلية: تقوم الحلقة بتكرار نفسها لكل "مستوى" داخل المصفوفة.
- مثال: يمكن استخدامها لطباعة قائمة "مواد الطالب" بالكامل دفعة واحدة.

خطوات تكرار مصفوفة الكلية الثلاثية (3D Array Iteration)

الخطوة الأولى: الدخول للمستوى الأول (الأقسام)

فتح الحلقة الأولى (المستوى الأول)

1. نبدأ بكتابة الكلمة المحجوزة `foreach` ثم نفتح قوساً هلالياً (، ونكتب داخل القوس اسم المصفوفة الكلية مسبقاً بعلامة الـ `$` مثل `$College`)، ثم نكتب كلمة `as`، يليها متغير يمثل "المفتاح" مسبقاً بعلامة الـ `$` مثل `$section`)، ثم علامة السهم `=>`، ثم متغير يمثل "القيمة" مسبقاً بعلامة الـ `$` مثل `$sub_sections`)، ونغلق القوس الهلالي (، ثم نفتح قوساً متعرجاً `{` للبدء.

المهمة: الوصول إلى الأوعية الكبرى داخل المصفوفة.

2. فتح الحلقة الثانية المتداخلة (المستوى الثاني)

داخل القوسين المتعرجين للحلقة الأولى، نكرر كتابة `foreach`، ولكن هذه المرة نستخدم المتغير الذي حمل قيمة المستوى الأول (مثل `$sub_sections`)، ثم نكتب كلمة `as`، يليها متغير جديد للمفتاح (مثل `$category`)، ثم سهم الربط `=>`، ثم متغير جديد للقيمة (مثل `$details`)، ونغلق القوس الهلالي (ونفتح قوساً متعرجاً جديداً `{`.

المهمة: الدخول إلى المصفوفات الفرعية (التصنيفات) داخل كل قسم.

3. فتح الحلقة الثالثة (المستوى الثالث والأخير)

داخل الحلقة الثانية، نكتب (foreach ونستخدم المتغير الذي حمل قيمة المستوى الثاني (مثل \$details، ثم كلمة as، يليها متغير يمثل مفتاح العنصر النهائي (مثل \$key)، ثم سهم الربط =>، ثم متغير يمثل القيمة الفعلية (مثل \$value)، ونغلق القوس الهلالي (ونفتح القوس المتعرج {}).

المهمة: الوصول إلى أعمق مستوى لاستخراج البيانات النهائية.

4. كتابة أمر العرض والربط

داخل الحلقة الثالثة، نكتب كلمة echo متبوعة بمتغير المفتاح (مثل \$key)، ثم نضع نقطة . (عامل الربط)، ثم نضع علامتي تنصيص بداخلها الرمز الذي نريده للفصل (مثل ":"، ثم نقطة أخرى .، ثم متغير القيمة (مثل \$value)، وننتهي بالفاصلة المنقوطة .;

المهمة: عرض "اسم العنصر" و "قيمته" بجانب بعضهما على الشاشة.

5. إغلاق البناء البرمجي

نقوم بكتابة قوس إغلاق متعرج {} لكل حلقة قمنا بفتحها، بحيث نغلق الحلقة الثالثة أولاً، ثم الثانية، ثم الأولى في النهاية.

المهمة: ضمان سلامة الكود ومنع حدوث خطأ في قواعد اللغة (Syntax Error)

```
foreach ($College as $section => $sub_sections)
{
    foreach ($sub_sections as $category => $details)
    {
        foreach ($details as $key => $value)
        {
            echo $key . ":" . $value ;
        }
    }
}
```